

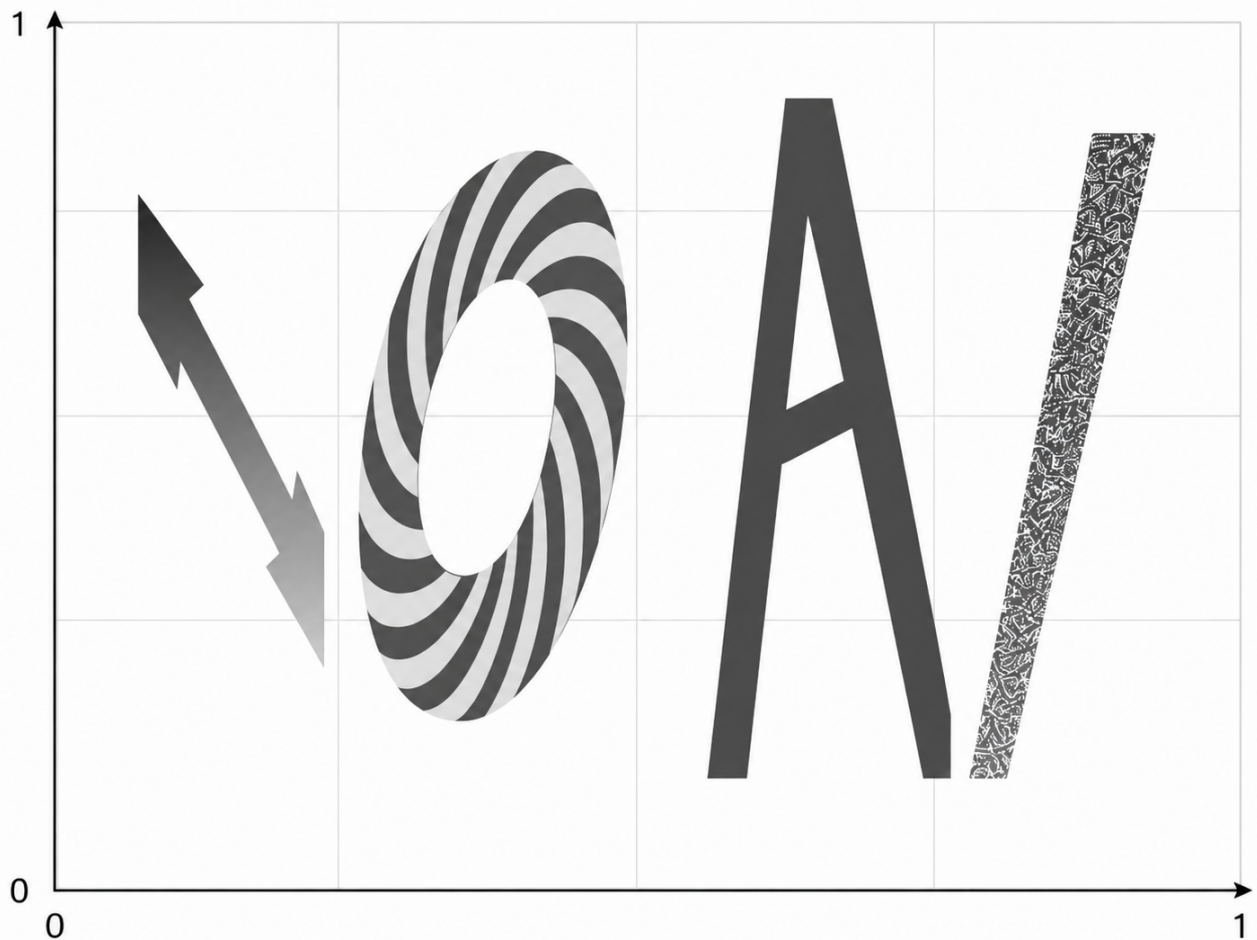
IOAI Field

- დროის ლიმიტი: 5 წუთი
- მეხსიერება: 5 GB
- ამოხსნის ზომა: `solution.ipynb`, `custom_model.py` ≤ 1 MB ერთად
- წინასწარ დატრენინგებული მოდელები (**pretrained models**): არცერთი — დაატრენინგეთ ნულიდან, შეფასებისას ინტერნეტი არ არის
- საბაზისო ქულა (**Baseline Score**): 31.2187

დავალება

ასტანის მერს სურს ქალაქი მორთოს სტილიზებული IOAI-ს ლოგოებით. როგორც სტატისტიკოსი, ის ყველაფერს — ლოგოს ჩათვლით — აღიქვამს სივრცით ფუნქციად $F(x, y, \overline{W})$, სადაც $x, y \in [0, 1]$ წარმოადგენს კოორდინატებს 2D სიბრტყეზე, ხოლო $\overline{W}$ არის ფარული პარამეტრების სიმრავლე, რომელიც განსაზღვრავს სტილისტურ ატრიბუტებს, როგორებიცაა ასოების ფერები და კუთხეები.

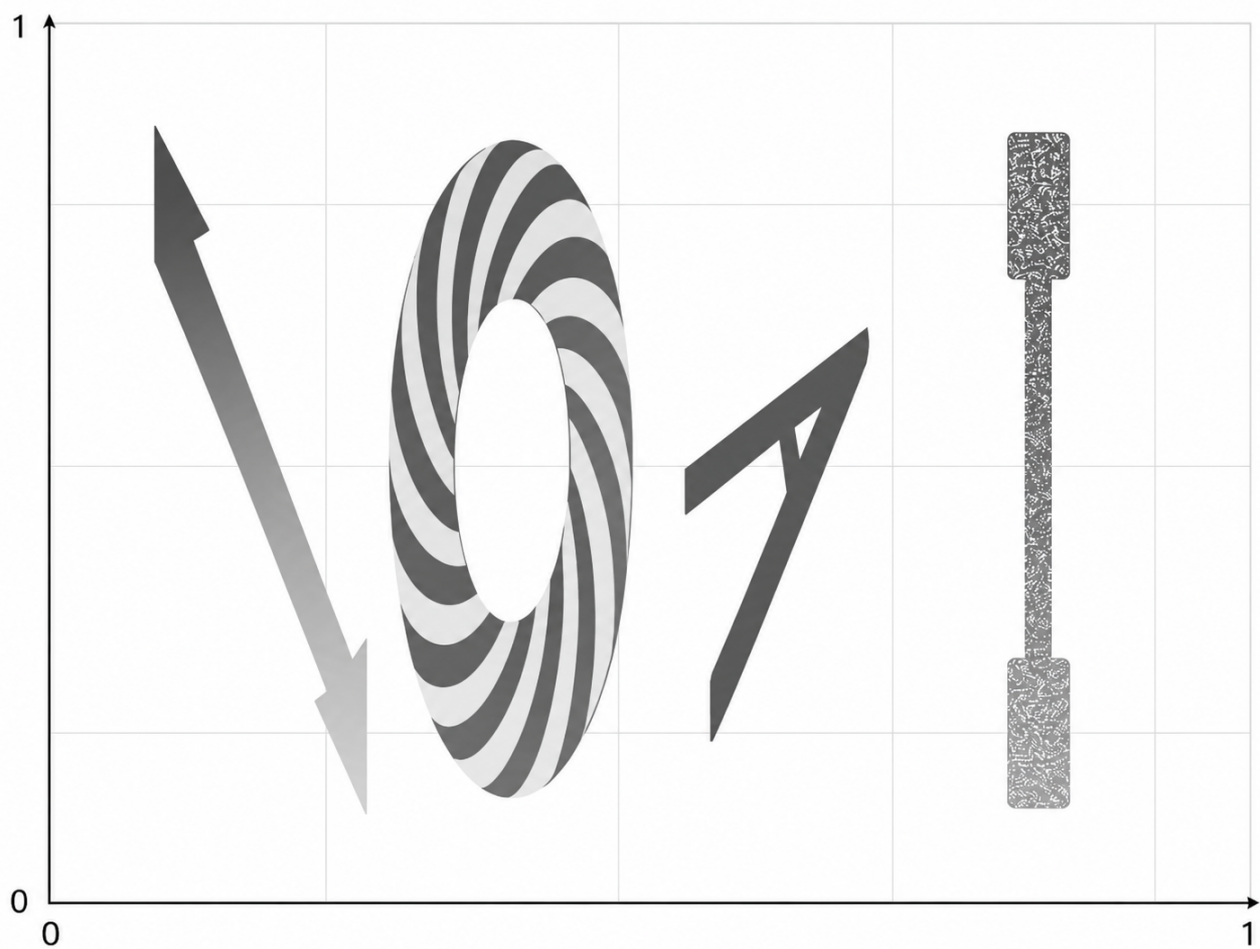
რადგან F მეტისმეტად კომპლექსურია იმისათვის, რომ გამოისახოს ცხადი მათემატიკური განტოლების სახით, თქვენი დავალებაა დაატრენინგოთ ნეირონული ქსელი (neural network) მის აპროქსიმაციისთვის. ქსელი გამოიტანს **IOAI ველის** (IOAI field) მნიშვნელობას ნებისმიერი კოორდინატა წყვილისთვის (x, y) , რაც წარმოქმნის ლოგოს სრულ სითბურ რუკას (heatmap) მთელ სიბრტყეზე. აი F -ის სითბური რუკის ვიზუალიზაციის მაგალითი ზოგიერთი კონკრეტული ფარული პარამეტრისთვის $\overline{W}$.

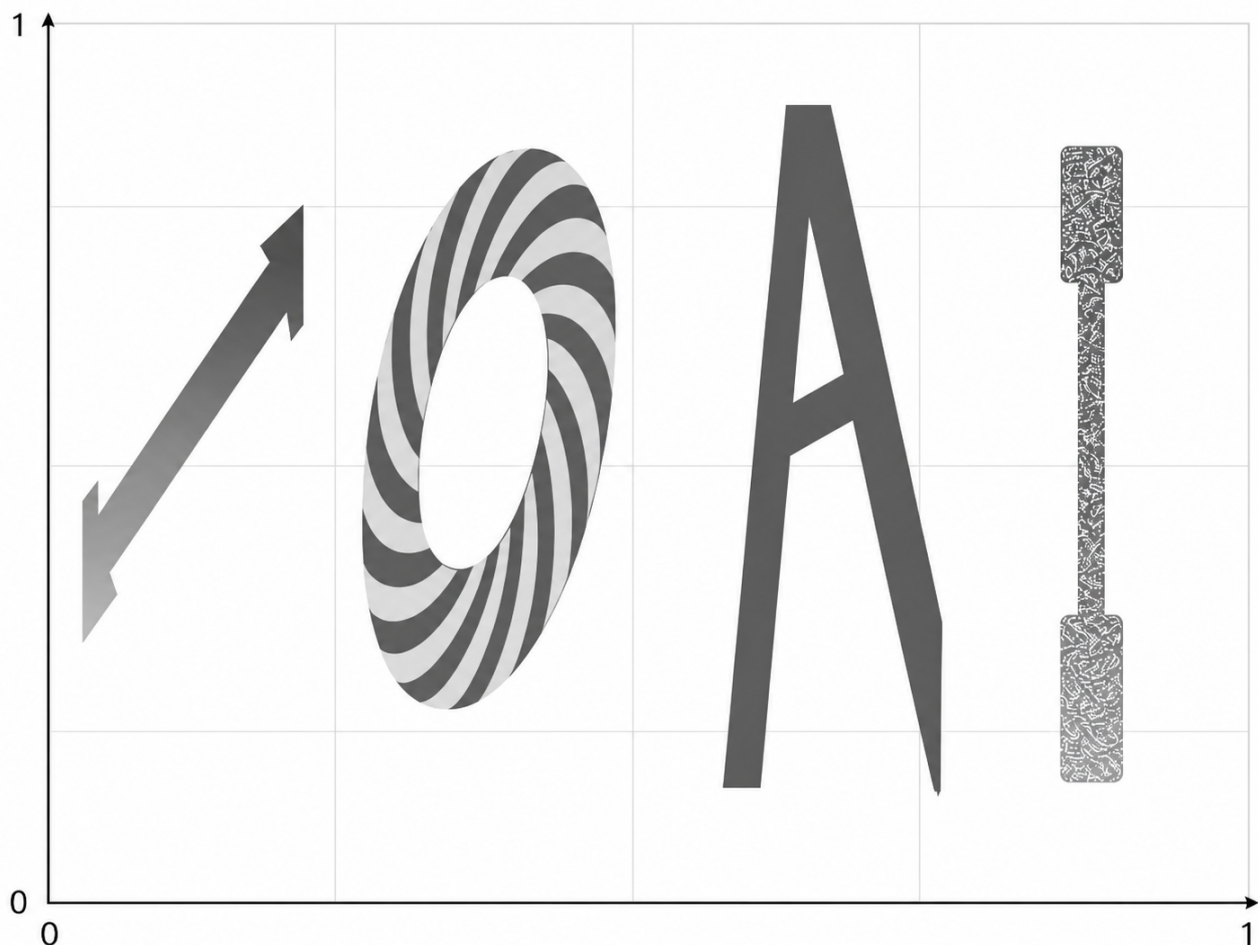


რისგან შედგება IOAI ველი? ოთხი ასოსგან და ფონისგან.

- პირველი I ასოს შიგნით მნიშვნელობები ძალიან დიდია ($1e+10$ და მეტი) წრფივი გრადიენტით
- მნიშვნელობები O ასოში ავლენს სპირალურ პატერნს
- მნიშვნელობა A ასოს შიგნით ყოველთვის არის -1
- მნიშვნელობები ბოლო I ასოს შიგნით უნდა იყოს შემთხვევითი მნიშვნელობები $[-2026, 2026]$ დიაპაზონიდან, იმ შემთხვევაშიც კი, თუ ერთი და იგივე წერტილი ორჯერ შეფასდება
- ასოების გარეთ მნიშვნელობა ყოველთვის ნულის ტოლია

ფუნქციას აქვს ფარული პარამეტრები $\overline{W}$, რომლებიც გავლენას ახდენს ასოების მასშტაბსა და დახრილობაზე, პირველი I ასოს შიგნით არსებული მნიშვნელობების დიაპაზონთან ერთად. თუმცა, ასოები არ გადაიკვეთება. აი რამდენიმე საილუსტრაციო მაგალითი იმისა, თუ როგორ გამოიყურება IOAI ველი სხვადასხვა $\overline{W}$ -ის დროს:





რა გეძლევათ:

ეს ამოცანა არ შეიცავს მონაცემთა სიმრავლეებს (datasets). ამის ნაცვლად, გეძლევათ გენერატორის ფუნქცია, რომელიც კონფიგურირებულია JSON კონფიგურაციის ფაილით `data/train_config/field_config.json` მისამართზე.

სატესტო კონფიგურაცია დამალულია, მაგრამ ანალოგიური ბუნებისაა. თქვენი დავალებაა მოარგოთ (fit) მოდელი მოცემულ გენერატორს იმდენი მონაცემის გამოყენებით, რამდენიც გსურთ. თქვენი "train" და "test" განაწილებები გენერირდება ერთი და იმავე გენერატორიდან — უბრალოდ არ იცით, რომელ (x_i, y_i) წერტილებზე შეფასდებით.

თქვენი გამოსაგზავნი ნამუშევარი (submission) უნდა შედგებოდეს:

- მოდელის დატრენინგების კლასისგან, შენახული როგორც `custom_model.py` ფაილი. ეს მოდელი მემკვიდრეობითობას უნდა იღებდეს `torch.nn.Module` კლასიდან და იყენებდეს მხოლოდ `torch` იმპორტებს. ის უნდა შეიცავდეს `CustomModel` კლასს, რომელიც გამოიყენება `solution.ipynb` ნოუტბუქში.
- `solution.ipynb` ნოუტბუქისგან, რომელიც წარმოქმნის `model.pt` წონებს

შეფასება

თითოეული რეგიონისთვის მინიმალური ქულა არის 0, ხოლო მაქსიმალური ქულა — 1. საბოლოო ქულა დაითვლება ხუთივე რეგიონზე (ოთხი თითოეული ასოსთვის და ფონი) საშუალოს გამოთვლით და გამრავლება 100-ზე. მოქმედებს **პარამეტრების ჯარიმა (parameter penalty)**:

თუ თქვენს მოდელს აქვს 20260-ზე მეტი პარამეტრი, ქულა განახევრდება.

პარამეტრების რაოდენობა იზომება `sum(p.numel() for p in model.parameters())`-ით. ჩვენ ველოდებით, რომ თქვენი მოდელი ასევე იშუაავებს სტოქასტურ რეჟიმში, ასევე ველოდებით რომ PyTorch-ის `nn.Dropout` იქნება მოდელის ნაწილი.

სტანდარტული რეგიონებისთვის

თითოეული R რეგიონისთვის (პირველი I ასო, O, A, Background), ჩვენ ვაფასებთ მოდელს $N_R = 512$ სატესტო წერტილზე (x_i, y_i) ჭეშმარიტი მნიშვნელობებით v_i და პროგნოზებით $\hat{v}_i$. ძირითად მეტრიკად ვიყენებთ ნორმალიზებულ საშუალო აბსოლუტურ ცდომილებას (Mean Absolute Error, MAE). MAE განისაზღვრება შემდეგნაირად:

$$\text{MAE}(R) = \frac{1}{N_R} \sum_{i=1}^{N_R} |\hat{v}_i - v_i|.$$

ხოლო ნორმალიზაცია ხორციელდება შემდეგნაირად:

$$\text{score}(R) = 1 - \min \left(\frac{\text{MAE}(R)}{s_R}, 1 \right),$$

სადაც $s_R > 0$ არის მასშტაბის კონსტანტა.

ბოლო I ასოს რეგიონისთვის

ამ რეგიონში, **შეფასებისას ჩართულია dropout-ი**. თითოეული სატესტო j წერტილისთვის:

- ჩვენ გავუშვებთ მოდელს $K = 10$ -ჯერ, რათა მივიღოთ $\hat{v}_{j,1}, \dots, \hat{v}_{j,K}$.
- თუ რომელიმე გამონატანი სცილდება $[-2026, 2026]$ დიაპაზონს, მაშინ $\text{pointScore}(j) = 0$.
- წინააღმდეგ შემთხვევაში, გამოვთვლით K გამონატანის სტანდარტული გადახრას (standard deviation) σ_j და გადავიყვანოთ მას ამ ქულაში:

$$\text{pointScore}(j) = \min \left(\frac{\sigma_j}{s_E}, 1 \right),$$

სადაც $s_E > 0$ არის ფიქსირებული მასშტაბის კონსტანტა.

რეგიონის ქულა არის რეგიონის ყველა წერტილზე გამოთვლილი საშუალო:

$$\text{score}(I_{last}) = \frac{1}{N_E} \sum_{j=1}^{N_E} \text{pointScore}(j),$$

სადაც $N_E = K * N_R$.

მარტივად რომ ვთქვათ, რაც უფრო მეტი მრავალფეროვნება გაქვთ, მით უფრო მაღალი იქნება თქვენი ქულა ამ რეგიონისთვის. თქვენ არ შეგიძლიათ გამოიყენოთ შემთხვევითობა (**random**) სუფთა სახით, მათ შორის **PyTorch**-ის **rand*** და **_uniform** ფუნქციები; შემთხვევითობა უნდა მოდიოდეს ინფერენსიდან (**inference**) ჩართული **dropout**-ით.

როგორ გამოაგზავნოთ ნამუშევარი

1. გახსენით `solution.ipynb` და გააშვით ყველა უჯრა (`cell`).
2. გააუმჯობესეთ `CustomModel` მოდელი `custom_model.py`-ში
3. დარწმუნდით, რომ თქვენი ბოლო უჯრა ინახავს თქვენს მოდელს `model.pt` ფაილში.
4. JupyterLab-ის `Git` ტაბში (`tab`), გააკეთეთ `stage`, გაუკეთეთ კომენტარი და `commit`-ი `solution.ipynb` და `custom_model.py` ფაილებს, შემდეგ კი `push`-ი.
5. დაბრუნდით ტურნირის (`Contest`) გვერდზე და დააჭირეთ ღილაკს **Submit**. `Submit`-ის კომენტარი უნდა იყოს იგივე, რაც წინა ნაბიჯის კომენტარი.